TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN

BÁO CÁO ĐỒ ÁN KỸ THUẬT LẬP TRÌNH PYTHON

XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN & MƯỢN TRẢ SÁCH THÔNG MINH

Giảng viên: ThS. Nghi Hoàng Khoa

Sinh viên thực hiện:

STT	Họ và tên sinh viên	MSVV	Phụ trách chính
1	Trần Ngọc Tâm	25410300	Book Catalog
2	Hồ Thiên Phúc	25410284	Dashboard & QA
3	Chu Văn Thơm	25410314	IAM & Infrastructure
4	Trần Bình Trọng	25410324	Borrowing Core
5	Lê Thanh Quốc	25410289	Fines & AI

Thành phố Hồ Chí Minh, tháng 09 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh, ngày … … tháng …… năm 20……

Người nhận xét

(Ký và ghi rõ họ tên)

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Thành viên	MSSV	Phụ trách	Công việc tiêu biểu	Ghi chú
1	Trần Ngọc Tâm	25410300	Book Catalog	Quản lý sách, tác giả, thể loại, nhà xuất bản; CRUD và tích hợp giao diện danh mục, thuyết trình	
2	Hồ Thiên Phúc	25410284	Dashboard Admin/User	Dashboard độc giả/admin, thống kê, thiết kế slide, Quay video demo	
3	Chu Văn Thơm	25410314	IAM & Infrastructure	Xác thực, phân quyền, refresh token, setup và cấu hình dự án, tích hợp và hạ tầng, tổng hợp báo cáo, thiết kế slide	
4	Trần Bình Trọng	25410324	Borrowing Core	Đặt trước, duyệt, mượn, trả, gia hạn, cập nhật tồn kho	
5	Lê Thanh Quốc	25410289	Fines & Recommendation	Mô hình Fine, AI embedding, semantic search, recommendation, thuyết trình	

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 8

1.1. Bối cảnh và lý do chọn đề tài 8

1.2. Mục tiêu của đề tài 9

1.3. Phạm vi và đối tượng sử dụng 9

1.3.1. Đối tượng sử dụng 9

1.3.2. Phạm vi phiên bản hiện tại 10

1.4. Công nghệ và thư viện sử dụng 11

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 13

2.1. Yêu cầu chức năng 13

2.1.1. Luồng nghiệp vụ mượn sách 15

2.2. Yêu cầu phi chức năng 16

2.3. Phân tích và thiết kế lập trình hướng đối tượng 17

2.3.1. Kiến trúc tổng thể 17

2.3.2. Class Diagram khái quát 17

2.3.3. Vai trò các lớp/service chính 18

2.3.4. Áp dụng các đặc trưng OOP 18

2.4. Thiết kế cơ sở dữ liệu 19

2.4.1. ERD 19

2.4.2. Mô tả các bảng 20

2.4.3. Các ràng buộc dữ liệu đáng chú ý 21

CHƯƠNG 3: TRIỂN KHAI VÀ CÀI ĐẶT MÃ NGUỒN 23

3.1. Cấu trúc tổ chức thư mục dự án 23

3.1.1. Kiến trúc backend 24

3.1.2. Kiến trúc frontend 24
3.2. Cài đặt lớp kết nối cơ sở dữ liệu 24
3.2.1. Đặc điểm thiết kế 25
3.2.2. Phòng chống SQL Injection 25
3.2.3. Xử lý transaction và ngoại lệ 25
3.3. Xử lý logic nghiệp vụ 26
3.3.1. Xác thực và quản lý phiên 26
3.3.2. Nghiệp vụ tồn kho và reservation 26
3.3.3. Gia hạn sách 27
3.3.4. Semantic Search 28
3.3.5. AI Indexing 28
3.3.6. Recommendation 28
3.3.7. Dashboard 29
3.3.8. Fine 29
3.4. Xử lý giao diện người dùng 30
3.4.1. Liên kết UI với backend 30
3.4.2. Routing và phân quyền 30
3.4.3. Quản lý state 31
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ DEMO GIAO DIỆN 32
4.1. Môi trường kiểm thử 32
4.2. Kịch bản kiểm thử 32
4.3. Hình ảnh demo giao diện 34
4.3.1. Màn hình đăng nhập 34
4.3.2. Trang chủ độc giả 35
4.3.3. Tìm kiếm sách và AI Semantic Search 36

4.3.4. Chi tiết sách....36
4.3.5. Danh sách đặt trước của độc giả....37
4.3.6. Dashboard Admin....38
4.3.7. Quản lý sách....38
4.3.8. Quản lý đặt trước....39
4.3.9. Quản lý mượn trả....39
4.3.10. Lịch sử mượn của độc giả....40
4.3.11. Gợi ý sách....40
CHƯƠNG 5: KẾT LUẬN & HƯỚNG PHÁT TRIỂN....42
5.1. Kết quả đạt được....42
5.2. Hạn chế của hệ thống....43
5.3. Hướng phát triển trong tương lai....43
TÀI LIỆU THAM KHẢO....45
PHỤ LỤC....46

DANH MỤC BẢNG

Bảng 1.1. Công nghệ và thư viện 11

Bảng 2.1. Yêu cầu chức năng 13

Bảng 2.2. Yêu cầu phi chức năng 16

Bảng 2.3. Các lớp chính 18

Bảng 2.4. Mô tả cơ sở dữ liệu 20

Bảng 3.1. Bảng routing và phân quyền 30

Bảng 4.1. Kịch bản kiểm thử 32

Bảng 5.1. Đối chiếu mục tiêu và kết quả 42

DANH MỤC HÌNH VẼ

Hình 2.1. Sơ đồ luồng nghiệp vụ mượn trả 15

Hình 2.2. Sơ đồ kiến trúc tổng thể 17

Hình 2.3. Kiến trúc phân tầng backend 17

Hình 2.4. Sơ đồ lớp thiết kế cơ sở dữ liệu hệ thống quản lý mượn sách 17

Hình 2.5. Sơ đồ mối quan hệ thực thể hệ thống quản lý thư viện. 19

Hình 3.1. Sơ đồ cấu trúc thư mục mã nguồn 23

Hình 3.2. Sơ đồ kiến trúc Backend 24

Hình 3.3. Sơ đồ kiến trúc Frontend 24

Hình 3.4. Sơ đồ luồng đăng nhập 26

Hình 3.5. Sơ đồ luồng nghiệp vụ tồn kho 27

Hình 3.6. Sơ đồ luồng đặt trước 30

Hình 4.1. Màn hình đăng nhập 34

Hình 4.2. Màn hình trang chủ độc giả 35

Hình 4.3. Màn hình trang tìm kiếm sách 36

Hình 4.4. Màn hình chi tiết sách 36

Hình 4.5. Màn hình sách đặt trước của độc giả 37

Hình 4.6. Màn hình tổng quan admin dashboard 38

Hình 4.7. Màn hình quản lý sách 38

Hình 4.8. Màn hình quản lý đặt trước 39

Hình 4.9. Màn hình quản lý mượn trả 39

Hình 4.10. Màn hình xem lịch sử mượn sách 40

Hình 4.11. Màn hình tổng quan của độc giả và gợi ý sách 40

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
ERD	Entity Relationship Diagram
JWT	JSON Web Token
RBAC	Role-Based Access Control
ORM	Object-Relational Mapping
SPA	Single Page Application
UI	User Interface
UX	User Experience
CI	Continuous Integration
AI	Artificial Intelligence
NLP	Natural Language Processing
UUID	Universally Unique Identifier
HTTP	Hypertext Transfer Protocol
CORS	Cross-Origin Resource Sharing

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Trong hoạt động quản lý thư viện, các công việc như theo dõi đầu sách, kiểm soát số lượng còn lại, quản lý người dùng, đặt trước, mượn - trả, gia hạn và thống kê thường liên quan tới nhiều nguồn dữ liệu và nhiều trạng thái nghiệp vụ. Nếu thực hiện thủ công hoặc sử dụng các hệ thống thiếu liên kết, cán bộ thư viện có thể gặp khó khăn trong việc theo dõi tồn kho thực tế, xác định phiếu mượn quá hạn, cập nhật tình trạng đặt trước, cũng như cung cấp khả năng tìm kiếm thuận tiện cho độc giả.

Bên cạnh bài toán quản lý nghiệp vụ truyền thống, trải nghiệm tìm kiếm sách cũng là một vấn đề đáng quan tâm. Tìm kiếm theo từ khóa chính xác có thể không đáp ứng tốt trường hợp người dùng chỉ nhớ nội dung, chủ đề hoặc nhu cầu đọc mà không biết tên sách. Vì vậy, nhóm xây dựng đề tài **"Hệ thống Quản lý Thư viện & Mượn Trả Sách Thông minh"**, kết hợp quy trình quản lý thư viện với tìm kiếm ngữ nghĩa và gợi ý sách dựa trên lịch sử mượn.

Đề tài phù hợp với môn **Kỹ thuật lập trình Python (IE221)** vì backend sử dụng Python làm ngôn ngữ chính, đồng thời áp dụng các nội dung quan trọng như:

- Thiết kế lớp và tổ chức mã nguồn theo mô-đun.
- Xử lý ngoại lệ và kiểm tra dữ liệu đầu vào.
- Làm việc với cơ sở dữ liệu quan hệ.
- Lập trình bất đồng bộ với async/await.
- Phân tách trách nhiệm giữa Router, Service và Repository.
- Kiểm thử tự động.
- Tích hợp thư viện AI chạy bằng Python.
- Quản lý dependency và môi trường thực thi.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát là xây dựng một ứng dụng web quản lý thư viện có thể phục vụ cả độc giả và quản trị viên, trong đó các chức năng chính được kết nối thành một luồng nghiệp vụ thống nhất.

Các mục tiêu cụ thể:

- Xây dựng cơ chế đăng ký, đăng nhập, đăng xuất và quản lý phiên người dùng.
- Phân quyền hai vai trò chính: USER và ADMIN.
- Quản lý danh mục sách, thể loại, tác giả và nhà xuất bản.
- Theo dõi tổng số lượng và số lượng sách khả dụng.
- Cho phép độc giả tìm kiếm, xem chi tiết và đặt trước sách.
- Cho phép quản trị viên duyệt hoặc từ chối yêu cầu đặt trước.
- Chuyển phiếu đặt trước đã duyệt thành phiếu mượn.
- Theo dõi lịch sử mượn trả, hạn trả và trạng thái quá hạn.
- Hỗ trợ gia hạn theo các điều kiện nghiệp vụ.
- Khôi phục tồn kho khi sách được trả.
- Cung cấp Dashboard cho độc giả và quản trị viên.
- Tích hợp Semantic Search bằng vector embedding.
- Gợi ý sách cá nhân hóa từ lịch sử mượn.
- Tổ chức mã nguồn theo kiến trúc phân tầng, dễ kiểm thử và bảo trì.

1.3. Phạm vi và đối tượng sử dụng

1.3.1. Đối tượng sử dụng

Độc giả (USER):

- Đăng ký và đăng nhập.
- Tìm kiếm và lọc sách.
- Xem chi tiết sách.

- Tạo và theo dõi phiếu đặt trước.
- Hủy phiếu đặt trước khi còn hợp lệ.
- Xem lịch sử mượn trả.
- Gia hạn phiếu mượn khi thỏa điều kiện.
- Xem Dashboard cá nhân.
- Nhận gợi ý sách.
- Xem hồ sơ và đổi mật khẩu.

Quản trị viên (ADMIN):

- Truy cập Dashboard quản trị.
- Quản lý sách, thể loại, tác giả, nhà xuất bản.
- Duyệt hoặc từ chối phiếu đặt trước.
- Xác nhận cho mượn từ phiếu đặt đã duyệt.
- Theo dõi danh sách phiếu mượn.
- Xác nhận trả sách.
- Sử dụng các API quản trị có RBAC bảo vệ.

1.3.2. Phạm vi phiên bản hiện tại

Phiên bản hiện tại tập trung vào nghiệp vụ thư viện và AI Search. Một số hạng mục đã có mô hình dữ liệu hoặc giao diện định hướng nhưng chưa được triển khai thành quy trình đầy đủ:

- Bảng **fines** đã tồn tại để lưu tiền phạt, nhưng chưa có workflow tự động sinh tiền phạt và xử lý thanh toán hoàn chỉnh.
- Chưa có cổng thanh toán trực tuyến.
- Chưa có background scheduler tự động xử lý reservation hết hạn; backend có API xử lý nhưng cần cơ chế gọi định kỳ bên ngoài nếu triển khai production.
- Embedding AI chưa tự động cập nhật mỗi khi metadata sách thay đổi.

- Chưa có giao diện quản trị danh sách user đầy đủ.
- Một số màn hình cài đặt/thông báo trên frontend mang tính định hướng phát triển và không được xem là chức năng hoàn chỉnh trong phạm vi demo.

1.4. Công nghệ và thư viện sử dụng

Bảng 1.1. Công nghệ và thư viện

Nhóm	Công nghệ	Vai trò
Ngôn ngữ backend	Python >= 3.12	Xây dựng API và logic nghiệp vụ
Backend framework	FastAPI >= 0.141	REST API bất đồng bộ
ORM	SQLAlchemy >= 2.0.52	Ánh xạ đối tượng - quan hệ, truy vấn async
Database driver	asyncpg >= 0.31	Kết nối PostgreSQL bất đồng bộ
Migration	Alembic >= 1.19	Quản lý phiên bản schema
CSDL	PostgreSQL	Lưu trữ dữ liệu nghiệp vụ
Vector DB extension	pgvector	Lưu vector 384 chiều, tìm kiếm tương đồng
Search extension	pg_trgm	Hỗ trợ GIN Trigram Index cho tên sách
Authentication	PyJWT, pwdlib/Argon2	JWT và băm mật khẩu
AI/NLP	sentence-transformers	Sinh embedding đa ngôn ngữ
Model AI	paraphrase-multilingual-MiniLM-L12-v2	Embedding 384 chiều
Frontend	React 19 + TypeScript	Xây dựng SPA
Build tool	Vite 8	Dev server và production build
Routing	React Router 8	Routing, lazy route và middleware

Nhóm	Công nghệ	Vai trò
Server state	TanStack React Query	Fetch/cache/invalidate dữ liệu API
Client state	Zustand	Quản lý auth state
HTTP client	Axios	Gọi API, interceptor và refresh queue
UI	Tailwind CSS 4, shadcn/ui	Thiết kế giao diện
Validation frontend	React Hook Form + Zod	Form và validation
Chart	Recharts	Biểu đồ Dashboard
Code quality backend	Ruff	Lint/format Python
Code quality frontend	ESLint, Prettier	Kiểm tra và định dạng TypeScript

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

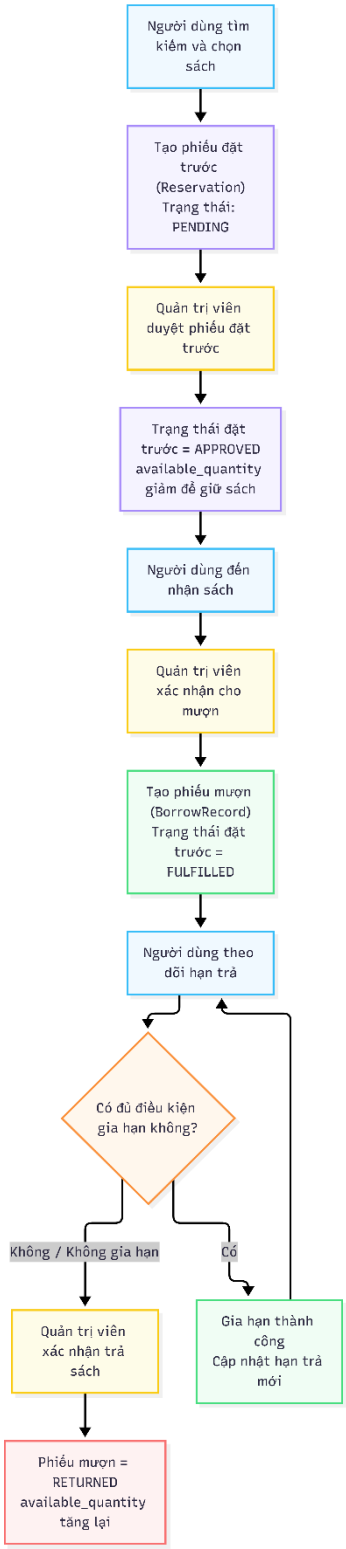
2.1. Yêu cầu chức năng

Bảng 2.1. Yêu cầu chức năng

Mã	Nhóm chức năng	Mô tả
FR-01	Đăng ký	Người dùng tạo tài khoản mới
FR-02	Đăng nhập	Xác thực email/mật khẩu và cấp access token
FR-03	Refresh session	Làm mới access token bằng HTTP-only refresh cookie
FR-04	Đăng xuất	Thu hồi refresh token và xóa cookie
FR-05	Đổi mật khẩu	Kiểm tra mật khẩu hiện tại, cập nhật mật khẩu mới và thu hồi phiên
FR-06	RBAC	Giới hạn API/route quản trị cho Admin
FR-07	Hồ sơ cá nhân	Lấy thông tin user hiện tại
FR-08	CRUD sách	Admin thêm, sửa, xóa, xem sách
FR-09	CRUD thể loại	Admin quản lý danh mục
FR-10	CRUD tác giả	Admin quản lý tác giả
FR-11	CRUD nhà xuất bản	Admin quản lý nhà xuất bản
FR-12	Tra cứu sách	Tìm theo tiêu đề, thể loại, tác giả và dữ liệu catalog
FR-13	Semantic Search	Tìm sách bằng mô tả tự nhiên
FR-14	Đặt trước	User tạo reservation
FR-15	Duyệt reservation	Admin duyệt reservation và giữ số lượng sách
FR-16	Từ chối reservation	Admin từ chối và ghi nhận lý do
FR-17	Hủy reservation	User/Admin hủy reservation hợp lệ
FR-18	Hết hạn reservation	Giải phóng tồn kho của reservation quá hạn

Mã	Nhóm chức năng	Mô tả
FR-19	Mượn từ reservation	Admin chuyển reservation đã duyệt thành borrow record
FR-20	Mượn trực tiếp	Backend hỗ trợ Admin tạo phiếu mượn trực tiếp
FR-21	Lịch sử mượn	User/Admin xem borrow records
FR-22	Gia hạn	User gia hạn khi phiếu đang mượn, chưa quá hạn và không có người khác chờ
FR-23	Trả sách	Admin xác nhận trả, cập nhật borrow item và khôi phục tồn kho
FR-24	Dashboard user	Thống kê nhanh, sách đang mượn, hoạt động, sách sắp đến hạn
FR-25	Dashboard admin	Thống kê tổng quan, mượn gần đây, phân bố thể loại, xu hướng
FR-26	Recommendation	Gợi ý sách từ lịch sử mượn, fallback theo độ phổ biến
FR-27	AI Indexing	Admin hoặc script tạo/cập nhật embedding cho catalog
FR-28	Lock/Unlock user	Backend hỗ trợ Admin khóa/mở khóa tài khoản
FR-29	Fine data model	Lưu dữ liệu phạt trong DB; workflow tự động chưa hoàn chỉnh

2.1.1. Luồng nghiệp vụ mượn sách



Hình 2.1. Sơ đồ luồng nghiệp vụ mượn trả

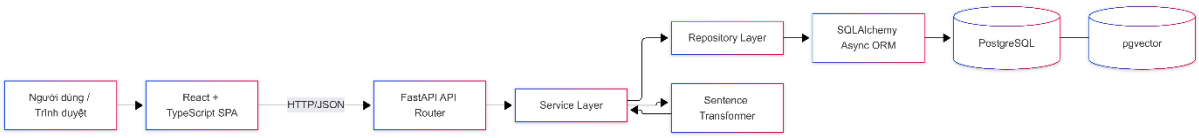
2.2. Yêu cầu phi chức năng

Bảng 2.2. Yêu cầu phi chức năng

Mã	Yêu cầu	Cách đáp ứng trong dự án
NFR-01	Bảo mật mật khẩu	Hash bằng Argon2
NFR-02	Quản lý phiên	JWT access token + refresh token rotation
NFR-03	Hạn chế XSS với refresh token	Refresh token đặt trong HTTP-only cookie
NFR-04	Phân quyền	RBAC ở API dependency và route middleware
NFR-05	Validation	Pydantic backend, Zod/React Hook Form frontend
NFR-06	Xử lý ngoại lệ	Domain exception, HTTP error chuẩn hóa
NFR-07	Không crash do request lỗi	FastAPI handlers và frontend error handling/toast
NFR-08	Tính toàn vẹn giao dịch	Async transaction trong BorrowingService/Repository
NFR-09	Hiệu năng I/O	AsyncSession + asyncpg
NFR-10	Tối ưu tìm kiếm	GIN Trigram Index và pgvector
NFR-11	Khả năng bảo trì	Layered Architecture, Feature-driven Frontend
NFR-12	Type safety	Python typing, Pydantic, TypeScript
NFR-13	Kiểm soát schema	Alembic migration
NFR-14	Chất lượng mã	Ruff, ESLint, Prettier, CI
NFR-15	Khả năng mở rộng	Service/Repository tách biệt, API versioning /api/v1
NFR-16	Trải nghiệm giao diện	SPA, lazy loading, responsive layout, loading/error states

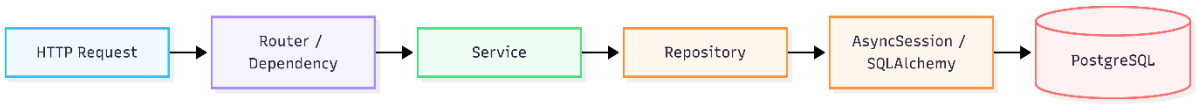
2.3. Phân tích và thiết kế lập trình hướng đối tượng

2.3.1. Kiến trúc tổng thể



Hình 2.2. Sơ đồ kiến trúc tổng thể

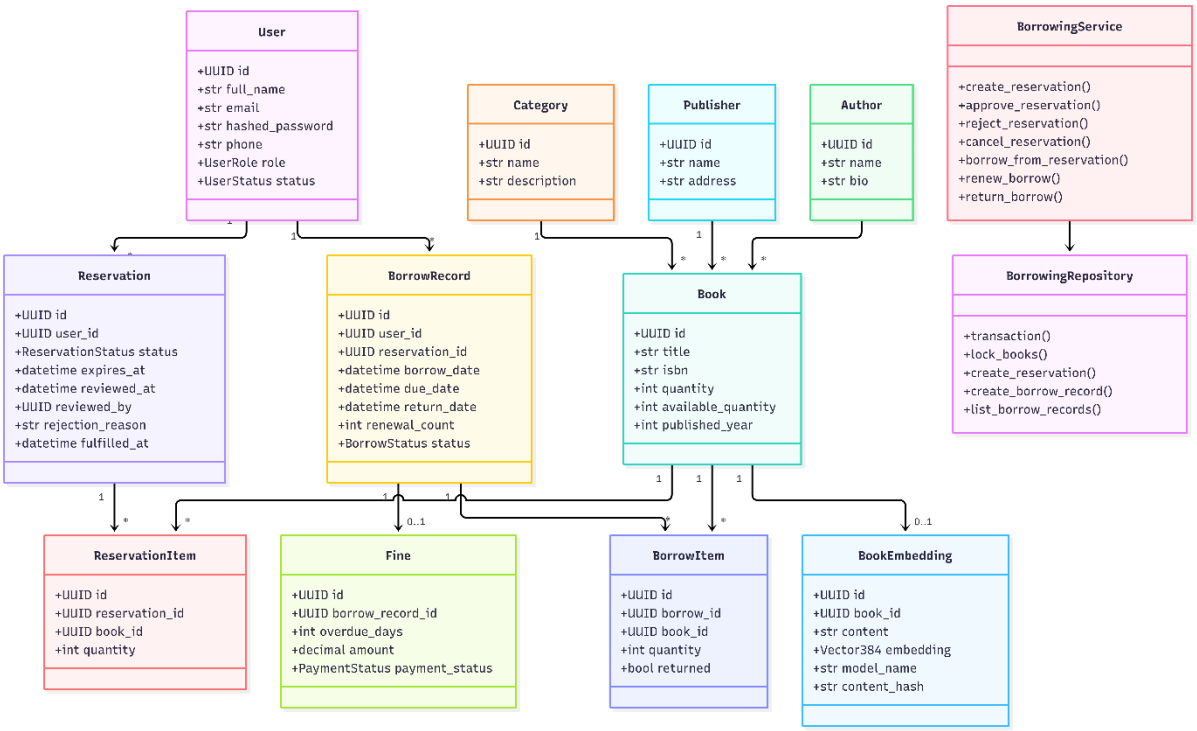
Backend áp dụng kiến trúc phân tầng:



Hình 2.3. Kiến trúc phân tầng backend

Cách tổ chức này giúp tách biệt trách nhiệm: Router không chứa truy vấn SQL, Service tập trung vào quy tắc nghiệp vụ, Repository phụ trách truy cập dữ liệu, Model mô tả cấu trúc lưu trữ và Schema quản lý dữ liệu request/response.

2.3.2. Class Diagram khái quát



Hình 2.4. Sơ đồ lớp thiết kế cơ sở dữ liệu hệ thống quản lý mượn sách

2.3.3. Vai trò các lớp/service chính

Bảng 2.3. Các lớp chính

Lớp/nhóm lớp	Vai trò
User	Biểu diễn tài khoản, vai trò và trạng thái
Book	Lưu metadata, tổng số lượng và số lượng khả dụng
Category, Author, Publisher	Dữ liệu danh mục phục vụ catalog
BookAuthor	Bảng liên kết nhiều-nhiều Book - Author
Reservation	Biểu diễn yêu cầu đặt trước và trạng thái duyệt
ReservationItem	Các sách nằm trong một reservation
BorrowRecord	Phiếu mượn, hạn trả, số lần gia hạn, trạng thái
BorrowItem	Chi tiết sách thuộc phiếu mượn
Fine	Mô hình dữ liệu tiền phạt
BookEmbedding	Lưu nội dung index và vector 384 chiều
RefreshToken	Lưu hash refresh token và trạng thái revoke
AuthService	Đăng ký, đăng nhập, cấp/refresh/logout token, đổi mật khẩu
BorrowingService	Điều phối nghiệp vụ reservation – borrow – renew – return
DashboardService	Tổng hợp dữ liệu dashboard
AIRecommendationService	Tính gợi ý sách cho user
BorrowingRepository	Truy vấn DB cho nghiệp vụ borrowing
Pydantic Schema	Validate request/response, tách API contract khỏi ORM entity

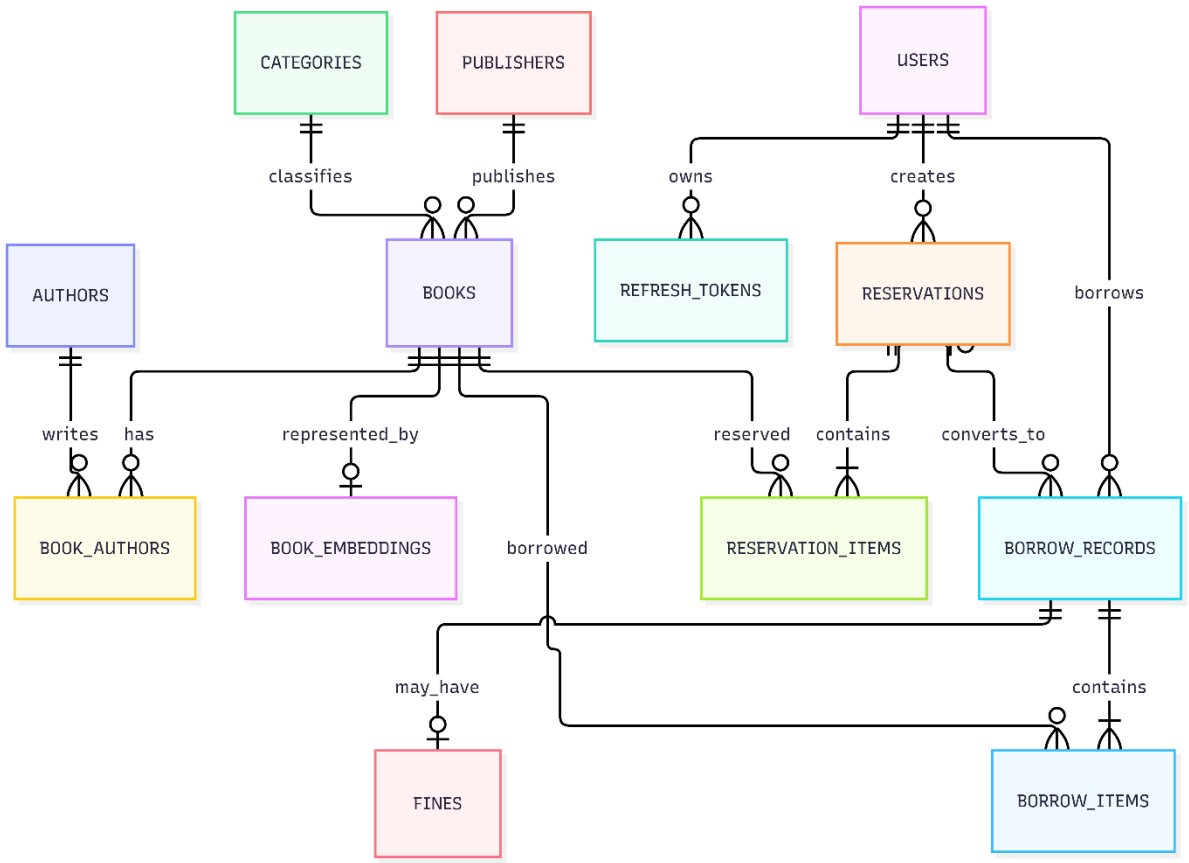
2.3.4. Áp dụng các đặc trưng OOP

- **Đóng gói:** logic nghiệp vụ nằm trong Service; truy cập DB được đóng gói trong Repository.

- **Kế thừa/tái sử dụng:** các SQLAlchemy Model tái sử dụng UUIDPkMixin, CreatedAtMixin, TimestampMixin.
- **Trừu tượng hóa:** Router chỉ biết interface của Service/Dependency, không thao tác trực tiếp với SQL ở các flow chính.
- **Composition:** Service nhận Repository thông qua dependency injection.
- **Enum domain:** trạng thái user, reservation, borrow và payment được biểu diễn bằng StrEnum, hạn chế magic string.
- **Data validation:** Pydantic Schema đóng vai trò DTO/API contract.

2.4. Thiết kế cơ sở dữ liệu

2.4.1. ERD



Hình 2.5. Sơ đồ mối quan hệ thực thể hệ thống quản lý thư viện.

2.4.2. Mô tả các bảng

Bảng 2.4. Mô tả cơ sở dữ liệu

Bảng	Các cột chính	Khóa/ràng buộc chính	Mục đích
users	id, full_name, email, hashed_password, phone, avatar_url, role, status	PK id; unique email	Tài khoản người dùng
refresh_tokens	id, user_id, hashed_token, expires_at, revoked	FK user_id; unique hashed_token	Quản lý refresh token
categories	Id, name, description	unique name	Thể loại sách
publishers	id, name, address	PK id	Nhà xuất bản
authors	id, name, bio	PK id	Tác giả
books	Id, title, description, isbn, category_id, publisher_id, quantity, available_quantity, published_year, cover_image_url	unique isbn; FK category/publisher; GIN Trigram Index title	Catalog và tồn kho
book_authors	book_id, author_id	PK kép; FK Book/Author	Quan hệ nhiều-nhiều
reservations	id, user_id, status, expires_at, reviewed_at, reviewed_by, rejection_reason, fulfilled_at	FK user/reviewer; index user+status	Phiếu đặt trước
reservation_items	id, reservation_id, book_id, quantity	unique reservation+book; FK	Chi tiết phiếu đặt

Bảng	Các cột chính	Khóa/ràng buộc chính	Mục đích
borrow_records	id, user_id, reservation_id, borrow_date, due_date, return_date, renewal_count, renewed_at, status	FK user/reservation; index user+status	Phiếu mượn
borrow_items	id, borrow_id, book_id, quantity, returned	unique borrow+book; FK	Chi tiết phiếu mượn
fines	id, borrow_record_id, overdue_days, amount, payment_status, paid_at	unique FK borrow record	Dữ liệu tiền phạt
book_embeddings	id, book_id, content, embedding, model_name, content_hash	unique book_id; Vector(384)	Tìm kiếm AI

2.4.3. Các ràng buộc dữ liệu đáng chú ý

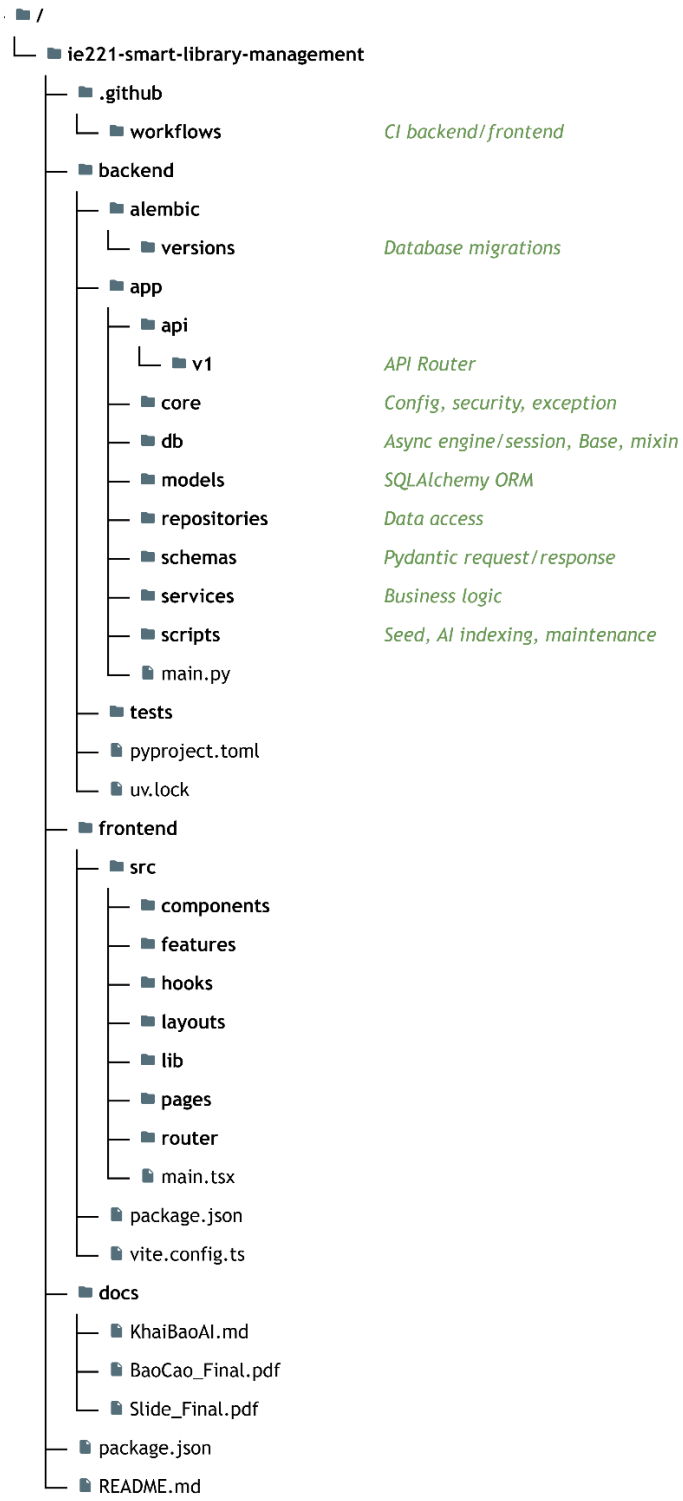
- **books.isbn** là duy nhất.
- **books.available_quantity** được dùng để thể hiện số lượng hiện có thể cấp cho người đọc.
- Book - Author sử dụng bảng trung gian **book_authors**.
- Mỗi đầu sách chỉ xuất hiện một lần trong cùng một reservation hoặc borrow record.
- Reservation có các trạng thái: **pending**, **approved**, **rejected**, **fulfilled**, **cancelled**, **expired**.
- Borrow có các trạng thái: **borrowing**, **returned**, **overdue**.
- Fine có trạng thái: **unpaid**, **paid**.

- **book_embeddings.embedding** có kích thước cố định 384.
- Alembic được dùng để đảm bảo thay đổi schema có lịch sử và có thể triển khai nhất quán giữa các môi trường.

CHƯƠNG 3: TRIỂN KHAI VÀ CÀI ĐẶT MÃ NGUỒN

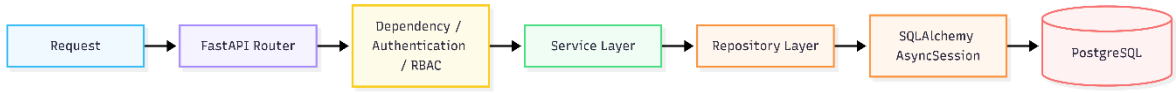
3.1. Cấu trúc tổ chức thư mục dự án

Dự án được tổ chức dưới dạng monorepo:



Hình 3.1. Sơ đồ cấu trúc thư mục mã nguồn

3.1.1. Kiến trúc backend



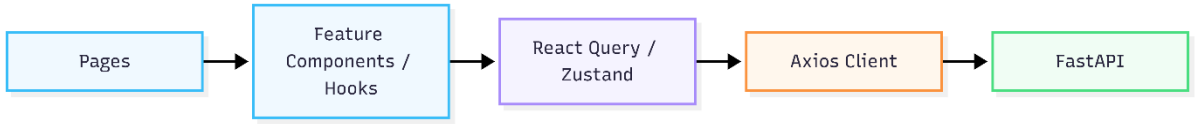
Hình 3.2. Sơ đồ kiến trúc Backend

Ưu điểm:

- Giảm phụ thuộc giữa API và database.
- Dễ viết unit test Service bằng repository giả.
- Quy tắc nghiệp vụ tập trung tại một lớp.
- Dễ thay đổi cách truy cập DB mà không sửa giao diện API.
- Hỗ trợ transaction trong các thao tác có nhiều bước.

3.1.2. Kiến trúc frontend

Frontend áp dụng Feature-driven Architecture:



Hình 3.3. Sơ đồ kiến trúc Frontend

Các route được lazy load. Route riêng tư sử dụng middleware **requireAuth**, route quản trị sử dụng **requireAdmin**. React Query quản lý server state và invalidate cache sau mutation.

3.2. Cài đặt lớp kết nối cơ sở dữ liệu

Backend sử dụng SQLAlchemy Async:

```
engine = create_async_engine(
    settings.DATABASE_URL,
    pool_pre_ping=True
)
```

```
AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False
)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

3.2.1. Đặc điểm thiết kế

- Chuỗi kết nối lấy từ biến môi trường **DATABASE_URL**.
- **AsyncSession** được cấp qua FastAPI dependency.
- Session tự đóng sau mỗi request.
- **pool_pre_ping=True** kiểm tra kết nối trước khi tái sử dụng connection.
- Repository nhận **AsyncSession** thay vì tự tạo connection.
- Alembic quản lý schema qua migration.

3.2.2. Phòng chống SQL Injection

Dự án chủ yếu thao tác dữ liệu bằng SQLAlchemy expression/ORM thay vì nối chuỗi SQL thủ công. Ví dụ về mặt nguyên tắc:

```
statement = select(Book).where(Book.id == book_id)
result = await session.execute(statement)
```

Giá trị **book_id** được bind thành parameter bởi SQLAlchemy, không được ghép trực tiếp vào câu SQL. Điều này giảm rủi ro SQL Injection.

3.2.3. Xử lý transaction và ngoại lệ

Những nghiệp vụ nhiều bước như duyệt đặt trước, tạo phiếu mượn hoặc trả sách cần đảm bảo toàn vẹn. **BorrowingService** sử dụng transaction từ Repository để các thay đổi trạng thái và tồn kho được commit/rollback cùng nhau.

Ví dụ luồng duyệt reservation:

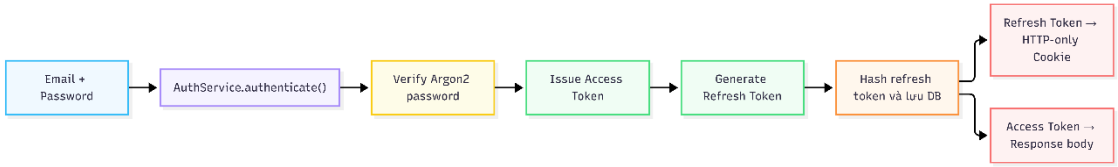
1. Lock reservation.
2. Lock các Book liên quan.
3. Kiểm tra số lượng khả dụng.
4. Giảm **available_quantity**.
5. Chuyển reservation sang **APPROVED**.
6. Gán thời hạn nhận sách.
7. Commit toàn bộ transaction.

Nếu một bước thất bại, transaction được rollback thay vì để DB ở trạng thái dở dang.

3.3. Xử lý logic nghiệp vụ

3.3.1. Xác thực và quản lý phiên

Luồng đăng nhập:



Hình 3.4. Sơ đồ luồng đăng nhập

Khi access token hết hạn, Axios interceptor gọi **/auth/refresh**. Backend xác thực cookie, rotate refresh token và trả access token mới. Các request đang chờ được tiếp tục sau khi refresh thành công.

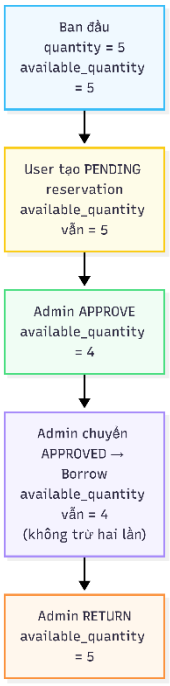
Khi đổi mật khẩu, các refresh token của người dùng được thu hồi và phía client xóa cache/auth state, sau đó quay về trang đăng nhập.

3.3.2. Nghiệp vụ tồn kho và reservation

Biến quan trọng:

```
quantity = tổng số bản sách
available_quantity = số bản hiện có thể cấp
```

Luồng tiêu biểu:



Hình 3.5. Sơ đồ luồng nghiệp vụ tồn kho

Điều này giúp tránh lỗi tồn kho âm hoặc giảm trùng.

3.3.3. Gia hạn sách

Các kiểm tra chính:

- Phiếu mượn phải thuộc user hiện tại.
- Phiếu phải đang ở trạng thái **BORROWING**.
- Chưa quá hạn.
- Chưa vượt **MAX_RENEWALS**.
- Không có độc giả khác đang chờ đầu sách.
- Nếu hợp lệ, cộng thêm **RENEWAL_DAYS** và tăng **renewal_count**.

Cấu hình hiện tại:

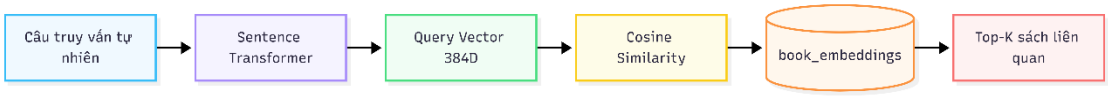
```
BORROW_DAYS = 14
RENEWAL_DAYS = 7
MAX_RENEWALS = 1
RESERVATION_HOLD_DAYS = 3
```

3.3.4. Semantic Search

Hệ thống sử dụng embedding model: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2**

Mỗi sách được tạo một embedding 384 chiều dựa trên metadata gồm thông tin như tiêu đề, tác giả, thể loại, nhà xuất bản, ISBN và mô tả.

Luồng tìm kiếm:



Ví dụ truy vấn: ***"sách giúp viết code sạch và dễ bảo trì"*** không yêu cầu người dùng biết chính xác tên sách nhưng hệ thống vẫn có thể tìm các đầu sách có nội dung liên quan tới lập trình và clean code.

3.3.5. AI Indexing

Khi index sách:

1. Tổng hợp metadata thành chuỗi nội dung.
2. Tính SHA-256 **content_hash**.
3. So sánh với embedding hiện có.
4. Nếu nội dung không đổi thì bỏ qua việc encode lại.
5. Nếu thay đổi thì sinh vector mới và upsert vào **book_embeddings**.

Cơ chế hash giúp giảm chi phí tính toán không cần thiết.

3.3.6. Recommendation

Recommendation sử dụng lịch sử mượn để xây dựng biểu diễn sở thích của người dùng, sau đó tìm các sách có embedding tương đồng. Nếu người dùng chưa có đủ lịch sử, hệ thống sử dụng chiến lược fallback dựa trên mức độ phổ biến.

Đây là điểm khác biệt giữa recommendation và search:

- Search bắt đầu từ **câu truy vấn hiện tại**.
- Recommendation bắt đầu từ **lịch sử hành vi của người dùng**.

3.3.7. Dashboard

Dashboard độc giả cung cấp dữ liệu như:

- Số sách đang mượn.
- Số sách quá hạn.
- Số reservation.
- Sách đang mượn.
- Hoạt động gần đây.
- Sách sắp đến hạn.
- Gợi ý sách.

Dashboard Admin tổng hợp:

- Các chỉ số toàn hệ thống.
- Phiếu mượn gần đây.
- Thống kê thể loại.
- Xu hướng mượn theo các khoảng 3 tháng, 6 tháng hoặc 12 tháng.

3.3.8. Fine

Database có bảng **fines** với:

- **borrow_record_id**
- **overdue_days**
- **amount**
- **payment_status**
- **paid_at**

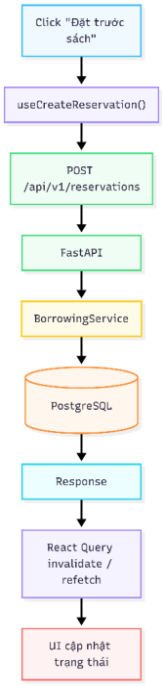
Tuy nhiên, trong phiên bản hiện tại chưa có Service/Repository/API đầy đủ để tự động sinh Fine từ phiếu mượn quá hạn và xử lý thanh toán. Vì vậy báo cáo chỉ xem Fine là **mô hình dữ liệu đã chuẩn bị cho chức năng mở rộng**, không xem là một workflow đã hoàn thiện.

3.4. Xử lý giao diện người dùng

3.4.1. Liên kết UI với backend

Frontend sử dụng Axios làm HTTP client. Các thao tác thay đổi dữ liệu được đóng gói trong hook/mutation của React Query.

Ví dụ luồng đặt trước:



Hình 3.6. Sơ đồ luồng đặt trước

3.4.2. Routing và phân quyền

Bảng 3.1. Bảng routing và phân quyền

Path	Quyền	Nội dung
/	Public	Trang chủ
/search	Public	Tìm kiếm sách
/books/:id	Public	Chi tiết sách
/login	Guest	Đăng nhập
/register	Guest	Đăng ký
/dashboard	Authenticated	Dashboard user

/book-reservation	Authenticated	Reservation của user
/borrow-history	Authenticated	Lịch sử mượn
/profile	Authenticated	Hồ sơ
/admin/dashboard	Admin	Dashboard admin
/admin/books	Admin	Quản lý sách
/admin/authors	Admin	Quản lý tác giả
/admin/publishers	Admin	Quản lý NXB
/admin/reservations	Admin	Duyệt đặt trước
/admin/borrows	Admin	Quản lý mượn/trả

3.4.3. Quản lý state

- **Zustand:** lưu trạng thái xác thực và access token.
- **React Query:** dữ liệu server, cache, refetch và mutation.
- **React Hook Form + Zod:** form đăng nhập, đăng ký và các validation phía client.
- **URL Search Params:** đồng bộ truy vấn/filter trong trang search.
- **Toast/Alert:** phản hồi thành công hoặc lỗi cho người dùng.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ DEMO GIAO DIỆN

4.1. Môi trường kiểm thử

Môi trường phát triển của dự án:

- Backend: Python >= 3.12.
- Frontend: Node.js >= 22.
- PostgreSQL có **pg_trgm** và **vector**.
- Frontend mặc định chạy tại **http://localhost:5173**.
- Backend mặc định chạy tại **http://localhost:8000**.
- API Docs tại **http://localhost:8000/docs**.

4.2. Kịch bản kiểm thử

Bảng 4.1. Kịch bản kiểm thử

TC	Chức năng	Đầu vào / thao tác	Kết quả mong đợi	Cách xác nhận
TC-01	Đăng nhập đúng	Email/password hợp lệ	Login thành công, vào hệ thống	UI + API
TC-02	Đăng nhập sai	Password không đúng	Trả lỗi, không tạo phiên	API/UI
TC-03	Refresh token	Access token hết hạn, refresh cookie hợp lệ	Nhận access token mới	API / interceptor
TC-04	RBAC	User truy cập admin route/API	Bị từ chối/chuyển hướng	UI + API
TC-05	CRUD sách	Admin tạo/sửa/xóa sách	DB và UI cập nhật	UI + API
TC-06	Search theo tên	Nhập một phần tiêu đề	Trả sách phù hợp	UI + API
TC-07	Semantic Search	Nhập mô tả tự nhiên	Trả sách có nội dung tương đồng	UI + AI API

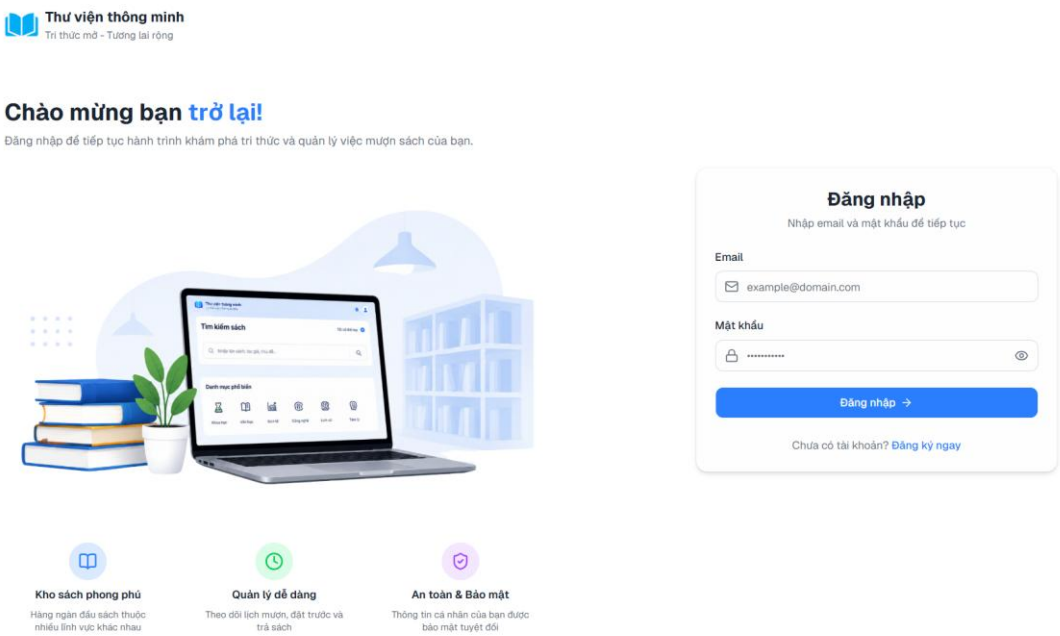
TC	Chức năng	Đầu vào / thao tác	Kết quả mong đợi	Cách xác nhận
TC-08	Tạo reservation	User chọn sách còn hàng	Reservation = Pending	UI + DB
TC-09	Duyệt reservation	Admin duyệt Pending	= Approved; tồn kho giảm	UI + DB
TC-10	Từ chối reservation	Admin nhập lý do hợp lệ	= Rejected; lưu lý do	UI + DB
TC-11	Mượn từ reservation	Admin xác nhận Approved	Borrow record được tạo; reservation = Fulfilled	UI + DB
TC-12	Không trừ tồn kho hai lần	Approve rồi borrow	available_quantity giữ nguyên ở bước borrow	Unit / manual test
TC-13	Gia hạn hợp lệ	Borrowing, chưa quá hạn, không có người chờ	due_date tăng, count tăng	UI + DB
TC-14	Chặn gia hạn khi có người chờ	Sách có waiting reservation	API từ chối gia hạn	Unit test
TC-15	Trả sách	Admin xác nhận trả	Returned, tồn kho tăng lại	UI + DB
TC-16	Quá hạn	due_date < now	Hiển thị trạng thái quá hạn phù hợp	Dashboard / Borrow API
TC-17	Dashboard User	User có dữ liệu lịch sử	Stats và danh sách đúng user	UI + API
TC-18	Dashboard Admin	Admin truy cập	Thống kê toàn hệ thống	UI + API
TC-19	Recommend ation	User có lịch sử mượn	Có danh sách đề xuất	AI API/UI
TC-20	Đổi mật khẩu	Mật khẩu cũ đúng	Đổi thành công, phiên bị thu hồi, yêu cầu login lại	UI + API

Bảng ghi kết quả nghiệm thu cuối

Nhóm kiểm tra	Lệnh / cách kiểm tra	Kết quả cuối
Backend migration	uv run alembic upgrade head	*PASS*
Backend lint	uv run ruff check .	*PASS*
Backend format	uv run ruff format --check .	*PASS*
Backend unit test	uv run python -m unittest discover -s tests -v	*Ran 19 tests in 0.094s* *OK*
Frontend lint	npm run lint	*PASS*
Frontend type check	npx tsc --noEmit	*PASS*
Frontend production build	npm run build	*PASS*
Manual E2E	Demo flow user → admin → user	*PASS*

4.3. Hình ảnh demo giao diện

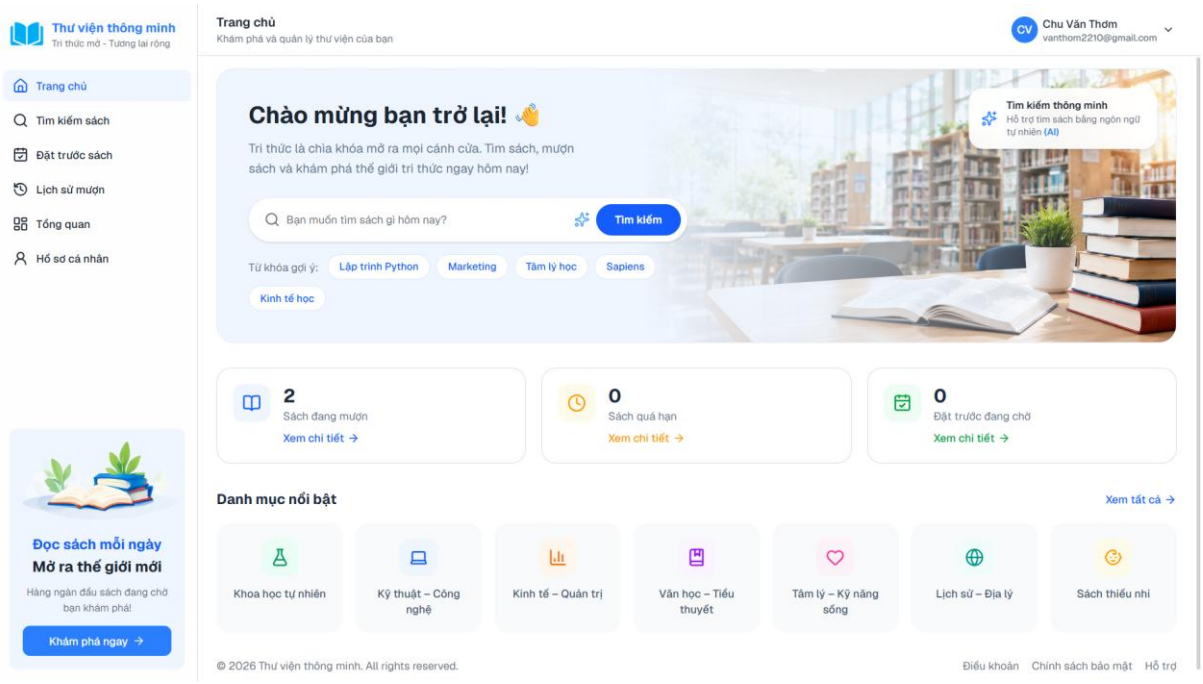
4.3.1. Màn hình đăng nhập



Hình 4.1. Màn hình đăng nhập

Chú thích: Màn hình đăng nhập xác thực email/mật khẩu và khởi tạo phiên người dùng.

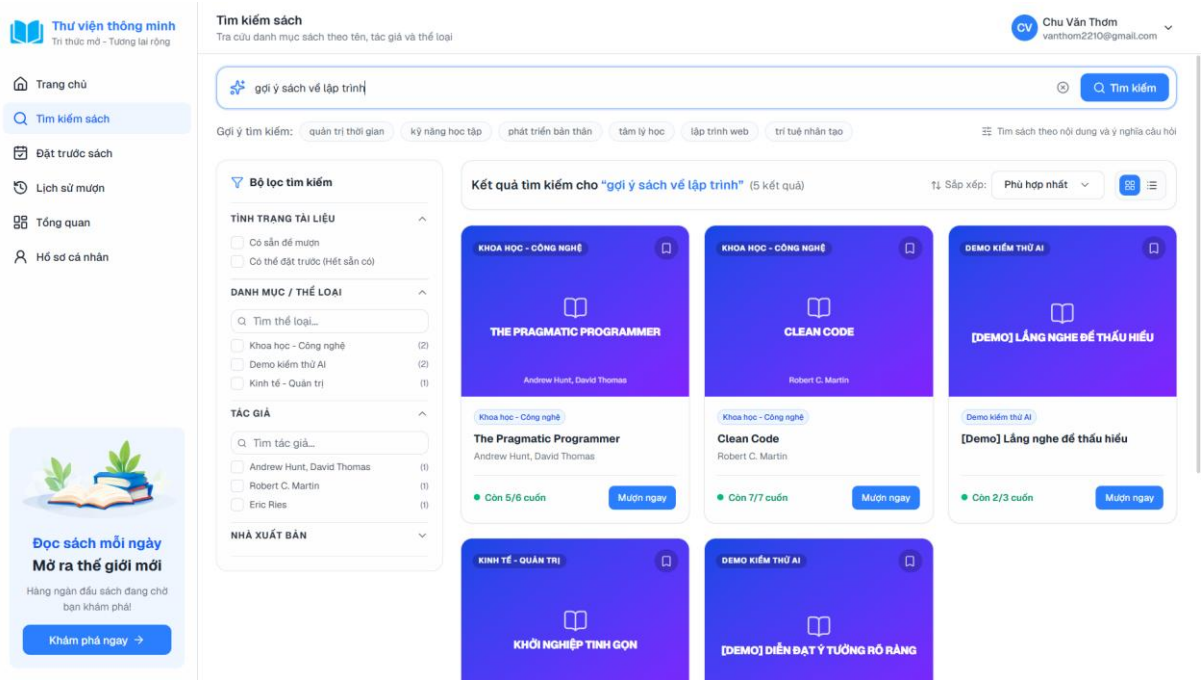
4.3.2. Trang chủ độc giả



Hình 4.2. Màn hình trang chủ độc giả

Chú thích: Trang chủ cung cấp truy cập nhanh tới tìm kiếm, thống kê cá nhân và các khu vực chức năng.

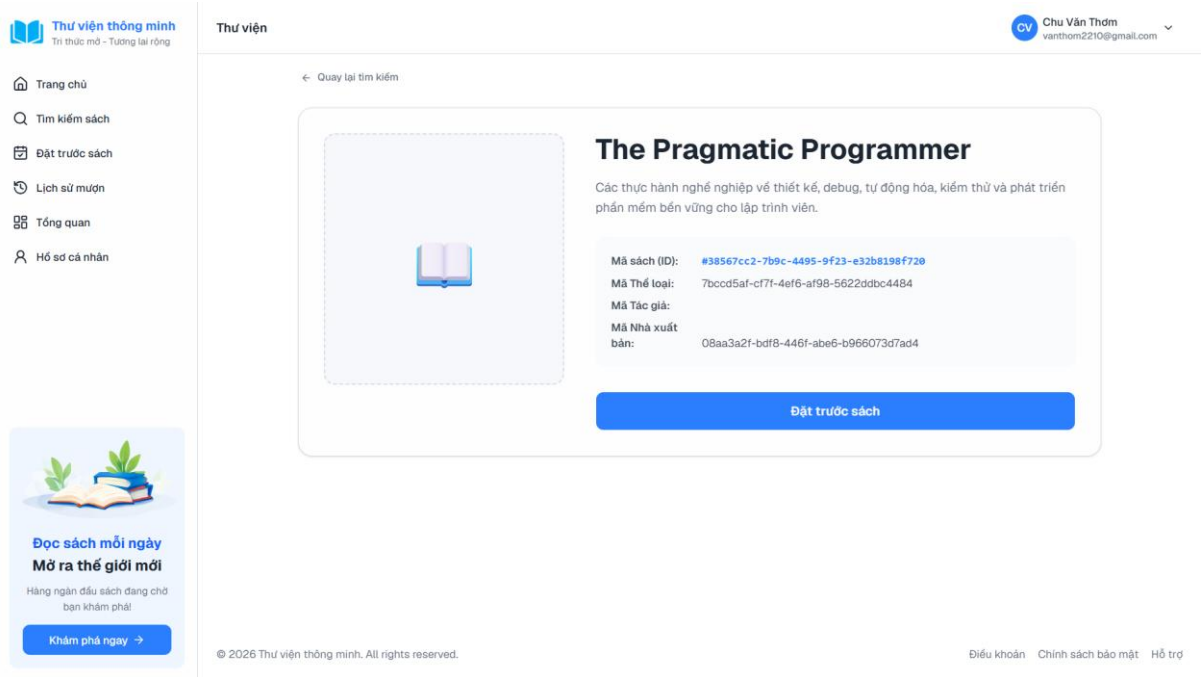
4.3.3. Tìm kiếm sách và AI Semantic Search



Hình 4.3. Màn hình trang tìm kiếm sách

Chú thích: Người dùng có thể tìm theo catalog hoặc mô tả nhu cầu bằng ngôn ngữ tự nhiên.

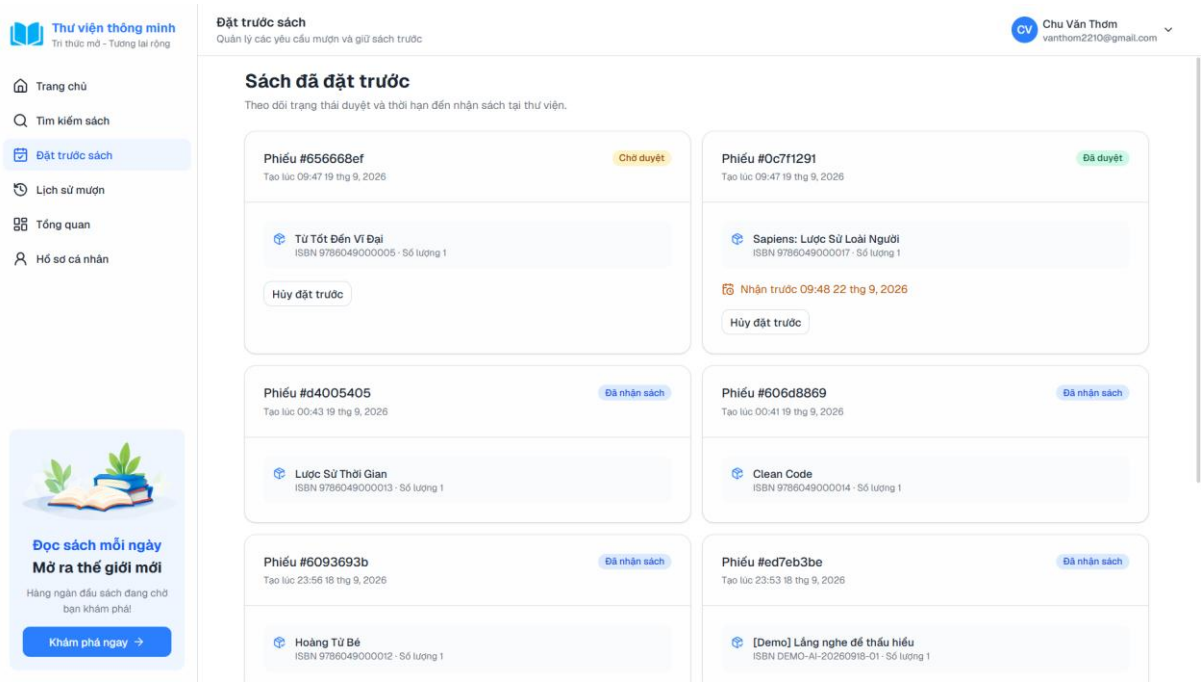
4.3.4. Chi tiết sách



Hình 4.4. Màn hình chi tiết sách

Chú thích: Hiển thị thông tin đầu sách và nút tạo reservation.

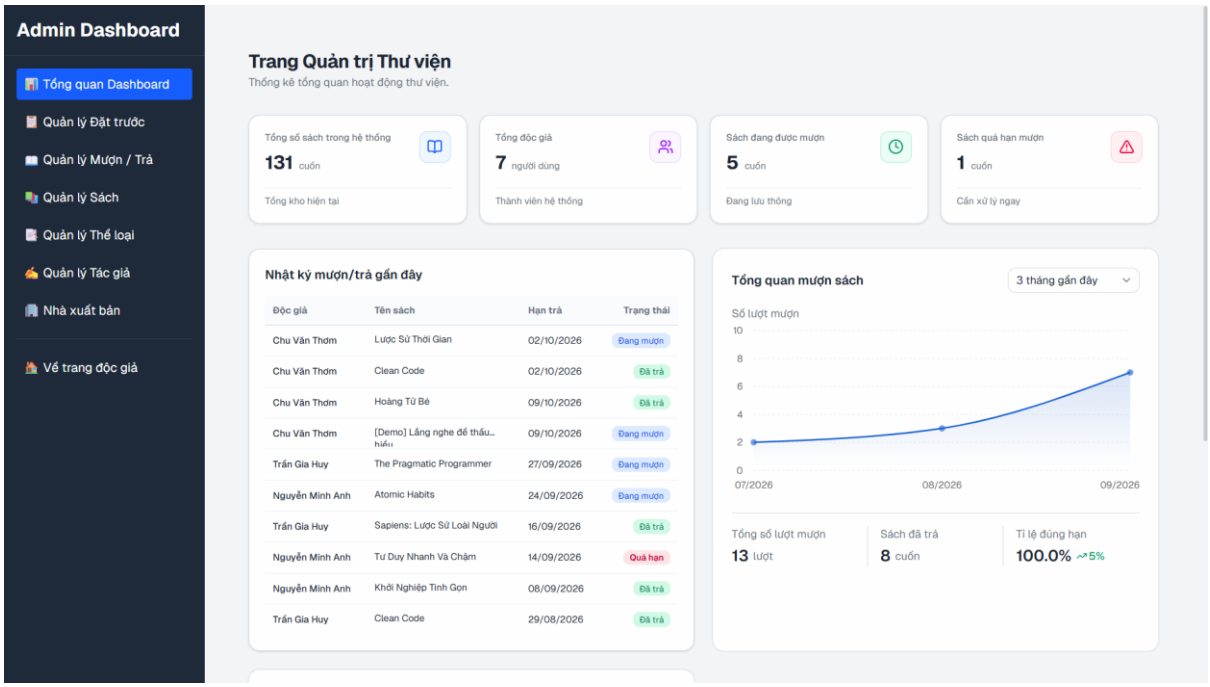
4.3.5. Danh sách đặt trước của độc giả



Hình 4.5. Màn hình sách đặt trước của độc giả

Chú thích: Theo dõi trạng thái Pending, Approved, Rejected, Fulfilled, Cancelled hoặc Expired.

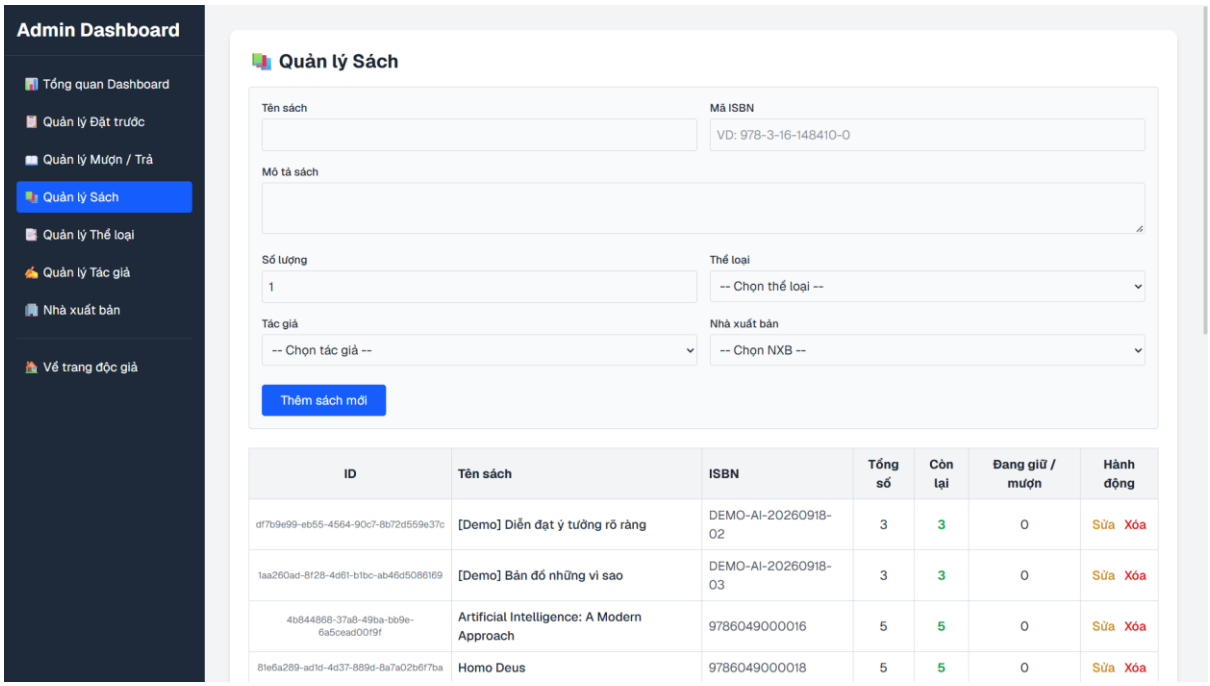
4.3.6. Dashboard Admin



Hình 4.6. Màn hình tổng quan admin dashboard

Chú thích: Tổng hợp số liệu và xu hướng hoạt động thư viện.

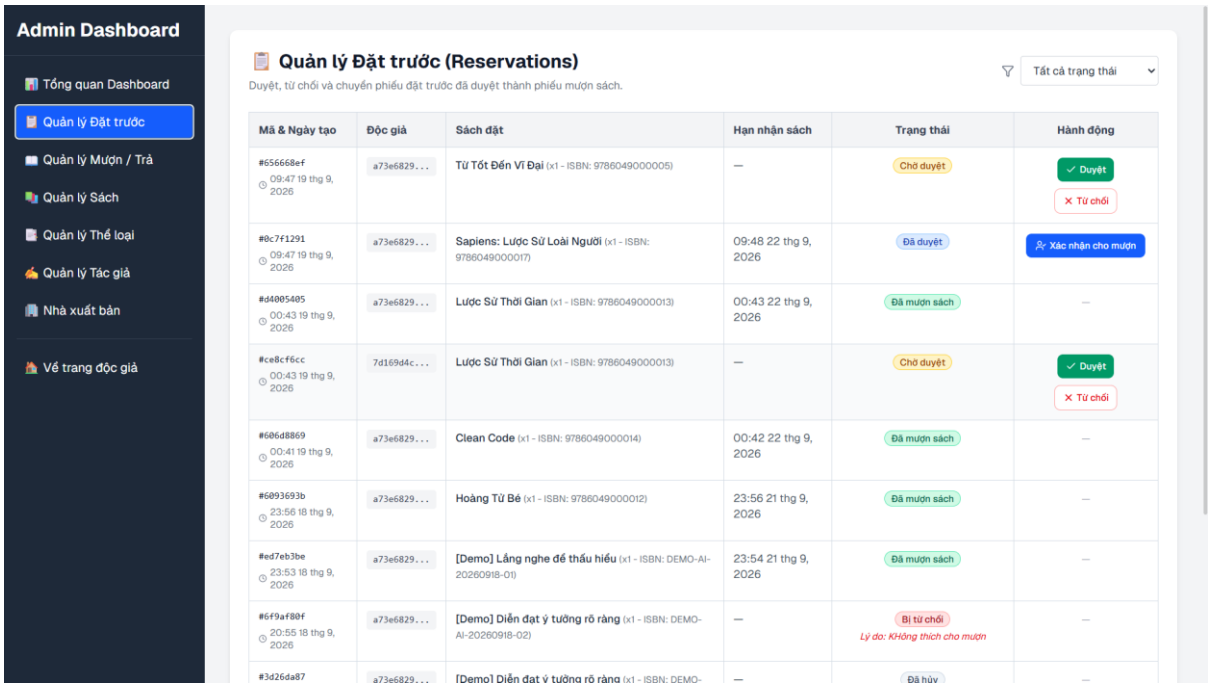
4.3.7. Quản lý sách



Hình 4.7. Màn hình quản lý sách

Chú thích: CRUD catalog và theo dõi số lượng tổng/số lượng khả dụng.

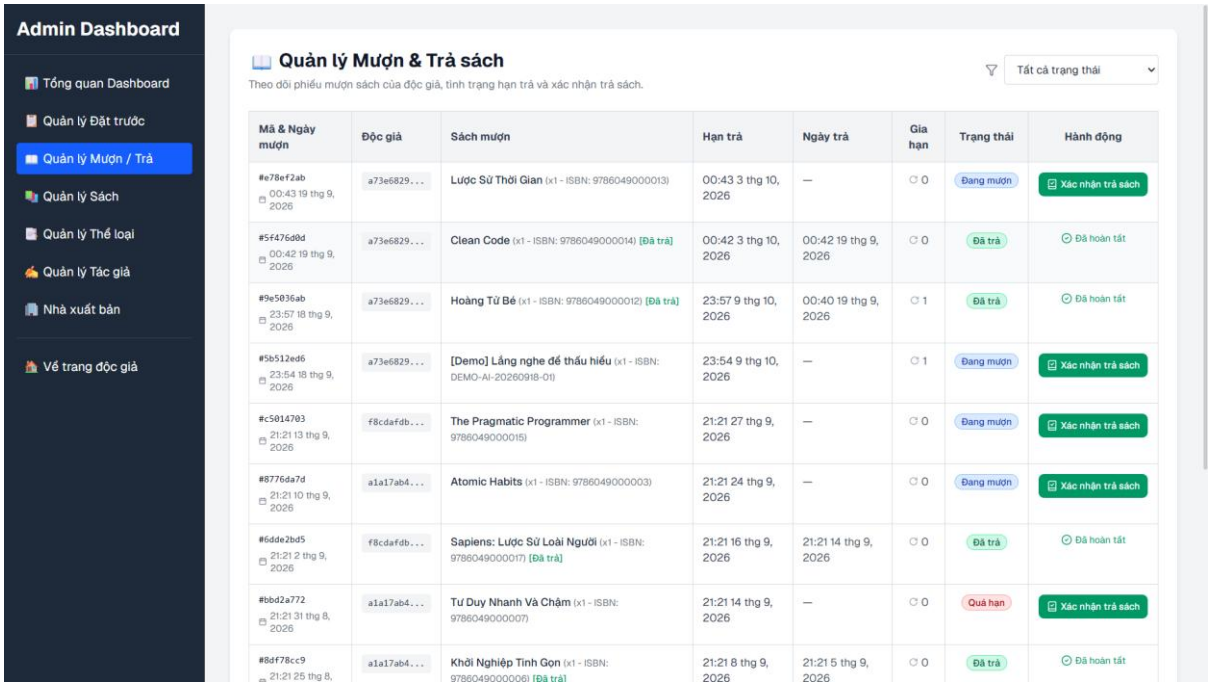
4.3.8. Quản lý đặt trước



Hình 4.8. Màn hình quản lý đặt trước

Chú thích: Admin duyệt/từ chối reservation và chuyển reservation đã duyệt thành phiếu mượn.

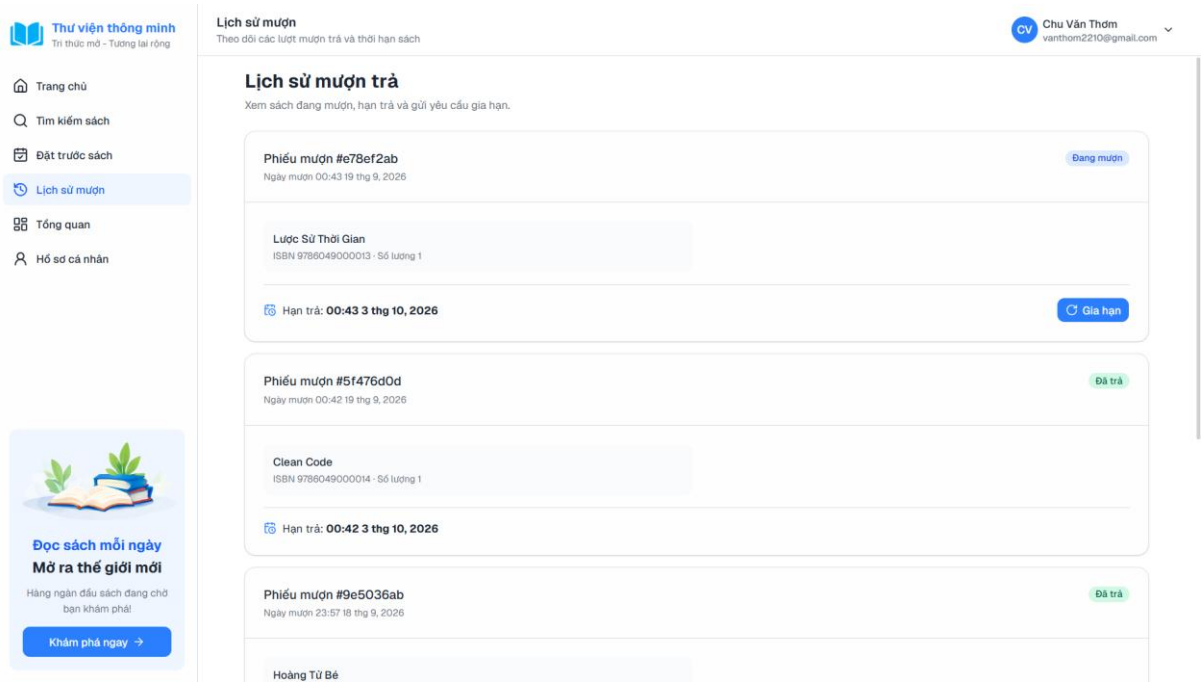
4.3.9. Quản lý mượn trả



Hình 4.9. Màn hình quản lý mượn trả

Chú thích: Theo dõi trạng thái mượn và xác nhận trả sách.

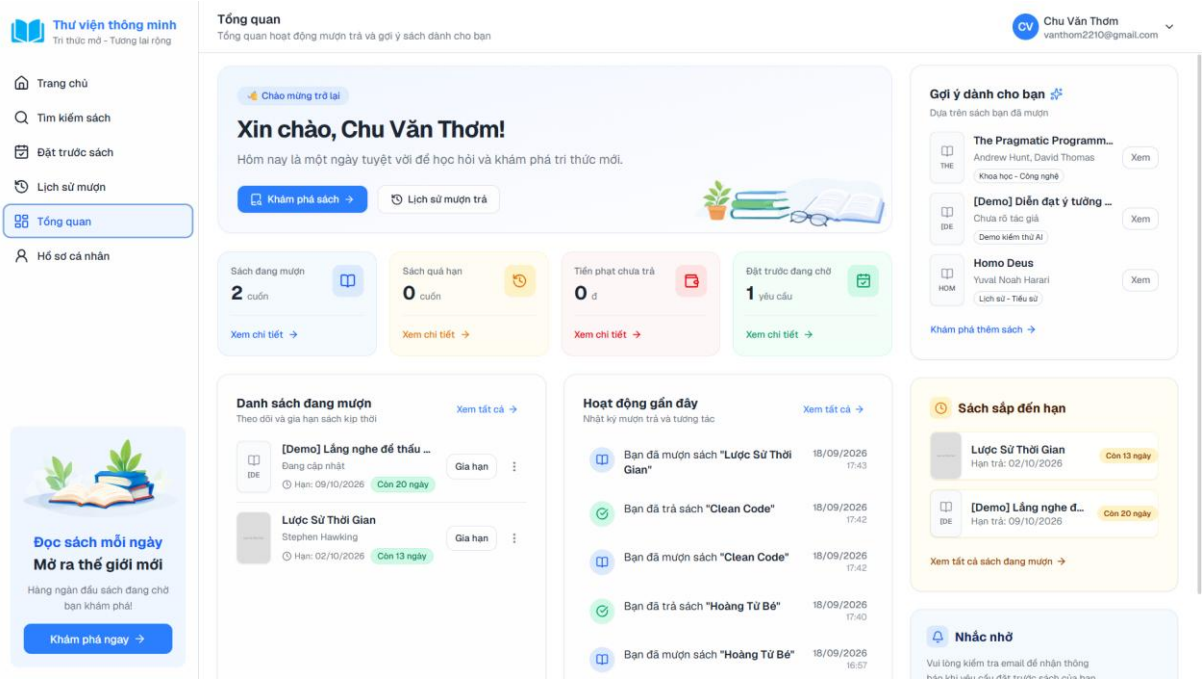
4.3.10. Lịch sử mượn của độc giả



Hình 4.10. Màn hình xem lịch sử mượn sách

Chú thích: User xem hạn trả, trạng thái và thực hiện gia hạn khi hợp lệ.

4.3.11. Gợi ý sách



Hình 4.11. Màn hình tổng quan của độc giả và gợi ý sách

Chú thích: Recommendation dựa trên lịch sử mượn và vector embedding.

CHƯƠNG 5: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Bảng 5.1. Đối chiếu mục tiêu và kết quả

Mục tiêu	Kết quả
Xây dựng backend Python	Hoàn thành bằng FastAPI
Kết nối PostgreSQL	Hoàn thành bằng SQLAlchemy Async + asyncpg
Quản lý schema	Có Alembic migrations
Xác thực	Có đăng ký, login, refresh, logout, đổi mật khẩu
Phân quyền	Có USER/ADMIN và RBAC
Catalog	Có CRUD Book/Category/Author/Publisher
Search	Có tìm kiếm catalog
Reservation	Có create/approve/reject/cancel/expire
Borrowing	Có mượn trực tiếp/từ reservation, list, return
Renewal	Có điều kiện gia hạn và số lần tối đa
Inventory	Tồn kho được cập nhật theo approve/return
Dashboard	Có Dashboard cho user và admin
AI Semantic Search	Có embedding 384D + cosine similarity
Recommendation	Có cá nhân hóa + popularity fallback
CI/code quality	Có Ruff, ESLint, TypeScript build/check và GitHub workflows
Fine	Mới hoàn thiện data model và phần đọc số liệu liên quan; workflow tự động chưa hoàn chỉnh

Tổng thể, hệ thống đã hình thành được một sản phẩm Client - Server có đầy đủ luồng nghiệp vụ thư viện chính và có điểm mở rộng AI rõ ràng.

5.2. Hạn chế của hệ thống

Các hạn chế hiện tại được xác định trực tiếp từ phạm vi mã nguồn:

1. **Fine chưa hoàn chỉnh:** có table/model nhưng chưa có workflow tự động tính và tạo Fine khi quá hạn, chưa có API thanh toán.
2. **Chưa có thanh toán trực tuyến:** chưa tích hợp cổng payment.
3. **AI index chưa đồng bộ tự động:** khi metadata sách thay đổi cần chạy indexing lại bằng script/API quản trị.
4. **Chưa có scheduler nền:** expire reservation và cleanup token cần được gọi bằng cron/scheduler khi triển khai production.
5. **Quản trị user chưa đầy đủ trên UI:** backend có lock/unlock nhưng chưa có trang danh sách user hoàn chỉnh.
6. **Notification chưa phải workflow hoàn chỉnh:** chưa có hệ thống gửi email/SMS thực tế được xác nhận trong backend.
7. **Quản lý ảnh bìa còn đơn giản:** Book lưu URL ảnh thay vì có media pipeline đầy đủ.
8. **AI model chạy local CPU:** phù hợp demo nhưng có thể tăng latency khi catalog lớn hoặc khi cần index số lượng lớn.
9. **Vector search chưa có index chuyên dụng cho quy mô lớn:** có thể cần HNSW/IVFFlat khi số lượng vector tăng.
10. **Test E2E trình duyệt chưa được tự động hóa:** hiện chủ yếu dựa vào unit test và manual regression.

5.3. Hướng phát triển trong tương lai

- Hoàn thiện FineService, FineRepository và API quản lý tiền phạt.
- Tự động tính tiền phạt từ số ngày quá hạn.
- Tích hợp thanh toán trực tuyến.
- Xây dựng notification service qua email hoặc push notification.
- Bổ sung scheduler định kỳ cho reservation expiry, token cleanup và job nghiệp vụ.
- Tự động re-index AI khi Book/Author/Category/Publisher thay đổi.
- Bổ sung vector index HNSW hoặc IVFFlat cho catalog lớn.

- Hoàn thiện trang Admin Users và quản lý vai trò.
- Tích hợp lưu trữ ảnh bìa chuyên dụng.
- Thêm audit log cho các thao tác quản trị.
- Thêm automated E2E test bằng Playwright/Cypress.
- Container hóa bằng Docker và triển khai CI/CD production.
- Cải thiện recommendation bằng thêm tín hiệu thể loại, tần suất, thời gian và đánh giá của độc giả.
- Xây dựng chức năng đánh giá/xếp hạng sách và danh sách yêu thích.

TÀI LIỆU THAM KHẢO

[1] Python Documentation. Đường dẫn: https://docs.python.org/3

[2] FastAPI Documentation. Đường dẫn: https://fastapi.tiangolo.com

[3] SQLAlchemy Documentation. Đường dẫn: https://docs.sqlalchemy.org

[4] Alembic Documentation. Đường dẫn: https://alembic.sqlalchemy.org

[5] PostgreSQL Documentation. Đường dẫn: https://www.postgresql.org/docs

[6] pgvector. Đường dẫn: https://github.com/pgvector/pgvector

[7] Pydantic Documentation. Đường dẫn: https://docs.pydantic.dev

[8] React Documentation. Đường dẫn: https://react.dev

[9] TypeScript Documentation. Đường dẫn: https://www.typescriptlang.org/docs

[10] Vite Documentation. Đường dẫn: https://vite.dev

[11] TanStack Query Documentation. Đường dẫn: https://tanstack.com/query/latest

[12] Zustand Documentation. Đường dẫn: https://zustand.docs.pmnd.rs

[13] Tailwind CSS Documentation. Đường dẫn: https://tailwindcss.com/docs

[14] shadcn/ui. Đường dẫn: https://ui.shadcn.com

[15] Sentence Transformers. Đường dẫn: https://www.sbert.net

[16] Model paraphrase-multilingual-MiniLM-L12-v2. Đường dẫn: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

PHỤ LỤC

Phụ lục A. Hướng dẫn cài đặt

A.1. Backend

```
cd backend

uv sync

# Tạo .env từ file mẫu và cấu hình:
# SECRET_KEY
# DATABASE_URL
# BACKEND_CORS_ORIGINS
# COOKIE_SECURE=false khi chạy HTTP local

uv run alembic upgrade head

uv run fastapi dev
```

Backend: http://localhost:8000

API Docs: http://localhost:8000/docs

A.2. Frontend

```
cd frontend

npm install
npm run dev
```

Frontend: http://localhost:5173

Phụ lục B. Lệnh kiểm tra chất lượng

Backend

```
cd backend

uv run alembic upgrade head
uv run alembic heads
uv run ruff check .
uv run ruff format --check .
uv run python -m unittest discover -s tests -v
```

Frontend

```
cd frontend

npm run lint
npx tsc --noEmit
npm run build
```

Phụ lục C. Chuẩn bị dữ liệu demo

```
cd backend

uv run alembic upgrade head
uv run python -m app.scripts.seed_demo
```

Tài khoản demo mặc định của seed:

Admin:

Email: demo.admin@example.com

Password: Demo123!@#

User:

Email: demo.minhanh@example.com

Password: Demo123!@#

User:

Email: demo.giahuy@example.com

Password: Demo123!@#

Phụ lục D. API chính

Authentication

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/change-password
```

User

```
GET   /api/v1/users/me
PATCH /api/v1/users/{id}/lock
```

```
PATCH /api/v1/users/{id}/unlock
```

Catalog

```
GET    /api/v1/books/
GET    /api/v1/books/search
POST   /api/v1/books/
PUT    /api/v1/books/{id}
DELETE /api/v1/books/{id}

GET/POST/PUT/DELETE /api/v1/categories/...
GET/POST/PUT/DELETE /api/v1/authors/...
GET/POST/PUT/DELETE /api/v1/publishers/...
```

Reservation / Borrow

```
POST   /api/v1/reservations
GET    /api/v1/reservations/me
GET    /api/v1/reservations
PATCH  /api/v1/reservations/{id}/approve
PATCH  /api/v1/reservations/{id}/reject
DELETE /api/v1/reservations/{id}
POST   /api/v1/reservations/expire

POST  /api/v1/borrow-records
POST  /api/v1/borrow-records/from-reservation/{id}
GET   /api/v1/borrow-records/me
GET   /api/v1/borrow-records
PATCH /api/v1/borrow-records/{id}/renew
PATCH /api/v1/borrow-records/{id}/return
```

AI

```
POST /api/v1/ai/search
POST /api/v1/ai/index-books
GET  /api/v1/ai/recommendations
```

Dashboard

```
GET /api/v1/dashboard/user/quick-stats
GET /api/v1/dashboard/user/borrowed-books
GET /api/v1/dashboard/user/recent-activities
GET /api/v1/dashboard/user/due-soon-books

GET /api/v1/dashboard/admin/stats
GET /api/v1/dashboard/admin/recent-borrows
GET /api/v1/dashboard/admin/category-stats
GET /api/v1/dashboard/admin/borrow-overview
```